Task 06 - Part 02: Question Answers

C# Fundamentals - Copy Constructors, Indexers & Lecture Recap

Q1. What is a copy constructor?

A copy constructor is a constructor that creates a new object by **copying the field values of an existing object of the same type**, instead of initializing fields from raw/individual values. C# doesn't generate one automatically the way some other languages do, so it's typically written explicitly, taking another instance of the same class or struct as its parameter and copying each field over. It's commonly used when you need an independent duplicate of an object - for example, to snapshot an object's current state before modifying it, or to hand out a copy that the caller can safely change without affecting the original. For a class containing reference-type fields, a copy constructor typically needs to decide between a shallow copy (copying the reference) or a deep copy (copying the referenced object too), depending on what behavior is actually needed.

Q2. What is an Indexer, when is it used, and what business cases require it?

An **indexer** is a special class member that allows instances of a class or struct to be accessed using array-like square-bracket syntax (e.g., myObject[0] or myObject["key"]), defined using the **this[]** syntax with get and/or set accessors. It's used whenever a class represents a logical collection or sequence of items internally (even if that data isn't literally a public array), and you want callers to interact with it using familiar, convenient indexing syntax rather than calling explicit methods like GetItem(i) or SetItem(i, value).

Business/real-world cases where an indexer is useful:

• A custom collection class (e.g., a ShoppingCart) where you want cart[0] to return the first item, hiding whether it's backed by a List, array, or something else internally.
• A configuration or settings class where config["ConnectionString"] reads a value by key, similar to a dictionary, but with extra validation or logic layered on top of the lookup.
• A Matrix or Grid class in a business application (e.g., a spreadsheet-like report) where matrix[row, col] is far more natural to read and write than matrix.GetValue(row, col).
• A class wrapping a database result set or cache, where record["ColumnName"] mirrors how developers already expect to access tabular or key-based data.

In general, an indexer is the right choice whenever a class's core responsibility is naturally something you'd want to "look up" or "index into", and using it makes calling code noticeably more intuitive to read and write.

Q3. Summarize the keywords learnt in the last lecture.

Based on the topics covered, the key concepts and keywords from the lecture were:

- **struct** - a lightweight value type; cannot inherit from another struct or class, but can implement interfaces.
- **class** - a reference type; supports inheritance and polymorphism.

• **Access modifiers** - private, internal, and public, controlling the scope and visibility of class members.

• **Encapsulation** - hiding internal state behind a controlled public interface (properties/methods) to protect data integrity.

• **Constructor overloading** - defining multiple constructors with different parameter lists to initialize an object in different ways.

• **Default vs parameterized constructors** - the implicit no-argument constructor versus custom constructors that accept initial values.

• **Copy constructor** - a constructor that initializes a new object by copying an existing instance's field values.

• **ToString() override** - customizing how an object's data is represented as readable text.

• **Value types vs reference types** - how structs (stack-allocated, copied by value) differ from classes (heap-allocated, copied by reference) in memory behavior when passed to methods.

• **Indexer** - the this[] syntax that lets a class be accessed using array-like square-bracket notation.